LabDesk Software Requirements Specification

Prototype requirements, workflows, and acceptance criteria

Student Chandhana S Gupta	**Institution** Sahrdaya College of Engineering and Technology
Student identifiers SR No. 225044 | Register No. SHR25CS093	**Prepared** 26 October 2026

Project note

LabDesk is a learning prototype prepared with AI coding assistance. The student should review and understand the implementation, confirm all details, and disclose assistance as required by course policy. Its features and results are described as implemented in the delivered prototype; no internship placement or external testing is claimed.

1. Purpose

This specification defines the current LabDesk learning prototype and the behavior expected from its student-facing equipment request flow and staff-facing review flow. It is a basis for implementation review and future work, not a claim that deferred production controls are already present.

2. Product Scope

LabDesk supports a single teaching laboratory. Students browse equipment, submit a loan request with a purpose and due date, and review its status. A lab coordinator reviews requests and records returns. The current prototype uses sample records and has optional MongoDB persistence when configured.

3. Users and Roles

User	Primary needs	Current prototype
Student	Find available equipment and request a loan with a return date.	Request form and equipment catalogue; no login.
Lab coordinator	Review requests and record check-ins.	Dashboard, request queue, and return view; not protected by server authorization.

4. Functional Requirements

ID	Requirement	Acceptance criteria
FR-01	Browse the equipment register.	Show item name, category, tag, location, and status; filter by availability and search text.
FR-02	Create a loan request.	Require student name, equipment, purpose, and due date; return a validation message when any field is missing; new request appears in the queue.
FR-03	Review a request.	Staff can change a pending request to Approved; the status is visible on the request row.
FR-04	Record a return.	An approved request can be marked Returned and is reflected in the interface.
FR-05	Report API readiness.	GET /api/health returns ok and database state (connected or demo-memory).
FR-06	Persist data when configured.	With MONGODB_URI set and the database reachable, equipment and requests use Mongoose models; otherwise demonstration data is held in memory.

5. User Stories

As a student, I want to find an available instrument so that I can plan a practical session. As a student, I want to submit the reason and due date so that the lab coordinator can review my request. As a coordinator, I want to approve and check in equipment so that the catalogue can reflect the loan lifecycle.

6. Data Requirements

Entity	Fields
Equipment	name (required), category (required), tag (required and unique in MongoDB), status (Available, On loan, Maintenance), location, timestamps.
Loan request	student, equipment, purpose, dueDate (required), status (Pending, Approved, Returned, Rejected), timestamps.

7. API Contract

Method and path	Purpose	Response / errors
GET /api/health	Read service readiness.	200 JSON with ok and database state.
GET /api/equipment	List equipment.	200 JSON array; server error uses a safe message.
POST /api/equipment	Add an asset.	201 created record; 400 if name, category, or tag is missing.
GET /api/requests	List loan requests.	200 JSON array.
POST /api/requests	Create request.	201 created record; 400 if a required field is missing.
PATCH /api/requests/:id/status	Update request status.	200 updated row; 400 for unsupported status; 404 for unknown id.

8. Quality Requirements

- Responsive layout should remain usable on mobile and desktop widths.
- Form controls must have visible labels; navigation and actions use semantic buttons.
- The API limits JSON bodies to 32 KB and validates required fields and allowed status values.
- Secrets belong in environment variables; no actual secret values should be committed.
- The current demo mode is non-persistent and must be labeled as such.

9. Assumptions and Limitations

The prototype uses fictional names and equipment records. It has no authentication, role enforcement, ownership checks, real-time concurrency controls, audit log, email notification, or production database configuration. The browser app remains a demo until a MongoDB connection and deployment variables are configured and the live workflow is verified.

10. Future Requirements

Add student and coordinator sign-in, role-protected API routes, ownership rules, pagination, audit events, maintenance reporting, and integration checks against a managed MongoDB database. Reassess privacy, security, and hosting costs before production use.